Implementation of digital differential analyzer (DDA) a line drawing algorithm

The Digital Differential Analyzer (DDA) is a simple line drawing method used in computer graphics.
It draws a straight line between two points by calculating intermediate points step by step.
The algorithm increments x and y values gradually from the start point to the end point.
It uses floating-point arithmetic for these calculations.
Because of this, the line appears smooth and continuous.
DDA is easy to understand and widely used for learning basic graphics concepts.

Advantages:

- Simple and easy to implement
- Works for all types of line slopes
- Produces smooth lines

Disadvantages:

- Uses floating-point operations, making it slower
- Rounding errors may reduce accuracy
- Less efficient than other algorithms like Bresenham's

C code

```
#include <stdio.h>
#include <math.h>

int main() {
   float x1, y1, x2, y2;
   float dx, dy, steps;
   float x_inc, y_inc;
   float x, y;
   int i;

   printf("Enter x1 y1: ");
   scanf("%f %f", &x1, &y1);

   printf("Enter x2 y2: ");
   scanf("%f %f", &x2, &y2);

   dx = x2 - x1;
   dy = y2 - y1;

   steps = (fabs(dx) > fabs(dy)) ? fabs(dx) : fabs(dy);

   x_inc = dx / steps;
   y_inc = dy / steps;

   x = x1;
   y = y1;

   printf("\nPoints plotted using DDA Algorithm:\n");

   for (i = 0; i <= steps; i++) {
      printf("(%.2f , %.2f)\n", x, y);
      x += x_inc;
      y += y_inc;
   }

   return 0;
}
```

Implementation of the Brenmans algorithm

Bresenham's Line Drawing Algorithm is an efficient method used in computer graphics to draw straight lines.
It determines which pixel should be plotted next using a decision parameter.
Unlike DDA, it uses only **integer calculations**, avoiding floating-point operations.
This makes the algorithm faster and more efficient.
It incrementally selects the nearest pixel to the actual line path.
Because of its speed and accuracy, it is widely used in raster graphics systems.

Advantages:

- Uses only integer arithmetic (fast execution)
- More accurate with minimal rounding errors
- Efficient and suitable for real-time graphics

Disadvantages:

- Slightly more complex to understand than DDA
- Needs different handling for different line slopes

Code

```
#include <stdio.h>
int main() {
   int x1, y1, x2, y2;
   int dx, dy, p;
   int x, y;

   printf("Enter x1 y1: ");
   scanf("%d %d", &x1, &y1);

   printf("Enter x2 y2: ");
   scanf("%d %d", &x2, &y2);

   dx = x2 - x1;
   dy = y2 - y1;

   p = 2 * dy - dx;
   x = x1;
   y = y1;

   printf("\nPoints plotted using Bresenham's Algorithm:\n");
   printf("(%d , %d)\n", x, y);

   while (x < x2) {
      x++;

      if (p < 0) {
         p = p + 2 * dy;
      } else {
         y++;
         p = p + 2 * (dy - dx);
      }

      printf("(%d , %d)\n", x, y);
   }

   return 0;
}
```

Implementations of the mid-point circle

The Mid-point Circle Algorithm is used to draw a circle in computer graphics.
It determines the position of pixels using a decision parameter.
The algorithm takes advantage of the symmetry of a circle to reduce calculations.
Instead of computing the whole circle, it calculates points for one part and reflects them.
It uses only integer arithmetic, making it efficient and fast.
Points are plotted in all **eight octants** of the circle.

Advantages:

- Fast due to integer calculations
- Reduces computation using symmetry
- Accurate and efficient for circle drawing

Disadvantages:

- Slightly complex to understand
- Mainly suitable for circles, not other shapes

C Code

```
 #include <stdio.h>

void plotPoints(int xc, int yc, int x, int y) {
   printf("(%d , %d)\n", xc + x, yc + y);
   printf("(%d , %d)\n", xc - x, yc + y);
   printf("(%d , %d)\n", xc + x, yc - y);
   printf("(%d , %d)\n", xc - x, yc - y);
   printf("(%d , %d)\n", xc + y, yc + x);
   printf("(%d , %d)\n", xc - y, yc + x);
   printf("(%d , %d)\n", xc + y, yc - x);
   printf("(%d , %d)\n", xc - y, yc - x);
}

int main() {
   int xc, yc, r;
   int x, y;
   int p;

   printf("Enter center (xc yc): ");
   scanf("%d %d", &xc, &yc);

   printf("Enter radius: ");
   scanf("%d", &r);

   x = 0;
   y = r;
   p = 1 - r;

   printf("\nPoints plotted using Mid-Point Circle Algorithm:\n");
   plotPoints(xc, yc, x, y);
```

```
    while (x < y) {
        x++;

        if (p < 0) {
            p = p + 2 * x + 3;
        } else {
            y--;
            p = p + 2 * (x - y) + 5;
        }

        plotPoints(xc, yc, x, y);
    }

    return 0;
}
```

Mid point ellipse circle

The **Mid-point Ellipse Algorithm** is used to draw an ellipse in computer graphics.
The ellipse is divided into **two regions** to simplify pixel plotting.
It uses **decision parameters** to choose the next pixel at each step.
The algorithm applies **symmetry** to reduce computations.
Only integer calculations are needed, making it efficient.
Points are mirrored across the axes to complete the ellipse quickly.

Advantages:

- Efficient due to integer arithmetic
- Reduces computation using symmetry
- Produces accurate ellipses

Disadvantages:

- More complex than circle drawing
- Mainly suited for ellipses, not other shapes

C Code

```
#include <stdio.h>
#include <math.h>
void plotPoints(int xc, int yc, int x, int y) {
   printf("(%d , %d)\n", xc + x, yc + y);
   printf("(%d , %d)\n", xc - x, yc + y);
   printf("(%d , %d)\n", xc + x, yc - y);
   printf("(%d , %d)\n", xc - x, yc - y);
}
int main() {
   int xc, yc, rx, ry;
   int x, y;
   double p1, p2;
 printf("Enter center (xc yc): ");
   scanf("%d %d", &xc, &yc);
   printf("Enter rx and ry: ");
   scanf("%d %d", &rx, &ry);
 x = 0;
   y = ry;

   p1 = (ry * ry) - (rx * rx * ry) + (0.25 * rx * rx);
  printf("\nPoints plotted using Mid-Point Ellipse Algorithm:\n");

   while ((2 * ry * ry * x) < (2 * rx * rx * y)) {
      plotPoints(xc, yc, x, y);
```

```
   if (p1 < 0) {
      x++;
      p1 = p1 + (2 * ry * ry * x) + (ry * ry);
   } else {
      x++;
      y--;
      p1 = p1 + (2 * ry * ry * x) - (2 * rx * rx * y) + (ry * ry);
   }
}

p2 = (ry * ry * (x + 0.5) * (x + 0.5)) +
    (rx * rx * (y - 1) * (y - 1)) -
    (rx * rx * ry * ry);
while (y >= 0) {
   plotPoints(xc, yc, x, y);

   if (p2 > 0) {
      y--;
      p2 = p2 - (2 * rx * rx * y) + (rx * rx);
   } else {
      x++;
      y--;
      p2 = p2 + (2 * ry * ry * x) - (2 * rx * rx * y) + (rx * rx);
   }
}
return 0; }
```

2d transformation

2D Transformations are used to modify the position, size, or orientation of objects in computer graphics.
The main types of transformations are translation, scaling, and rotation.
Translation moves an object from one location to another without changing its shape.
Scaling increases or decreases the size of an object relative to a reference point.
Rotation turns the object around a fixed point, usually the origin or its center.
These transformations are essential for animating, modeling, and manipulating graphics.

Advantages:

- Simple and widely used in graphics systems
- Allows easy movement, resizing, and rotation of objects

Disadvantages:

- Complex transformations may require combining multiple operations
- Precision errors can occur with repeated transformations

Code

```
#include <stdio.h>
#include <math.h>

int main() {
   int choice;
   float x, y, tx, ty, sx, sy, angle;
   float x_new, y_new;

   printf("Enter original point (x y): ");
   scanf("%f %f", &x, &y);

   printf("\n2D Transformations Menu:");
   printf("\n1. Translation");
   printf("\n2. Scaling");
   printf("\n3. Rotation");
   printf("\nEnter your choice: ");
   scanf("%d", &choice);

   switch (choice) {

      case 1: // Translation
         printf("Enter translation factors (tx ty): ");
         scanf("%f %f", &tx, &ty);
         x_new = x + tx;
```

```
        y_new = y + ty;
        break;

    case 2: // Scaling
        printf("Enter scaling factors (sx sy): ");
        scanf("%f %f", &sx, &sy);
        x_new = x * sx;
        y_new = y * sy;
        break;

    case 3: // Rotation
        printf("Enter angle (in degrees): ");
        scanf("%f", &angle);
        angle = angle * (3.14159 / 180); // convert to radians
        x_new = x * cos(angle) - y * sin(angle);
        y_new = x * sin(angle) + y * cos(angle);
        break;

    default:
        printf("Invalid choice!");
        return 0;
}

printf("\nNew point after transformation: (%.2f , %.2f)", x_new, y_new);

return 0;}
```

Implementation of 3D transformation

3D Transformations are used to modify objects in three-dimensional space.
They include **translation, scaling, and rotation**.
Translation moves an object to a new position in 3D space.
Scaling changes the size of the object along one or more axes.
Rotation turns the object around an axis (x, y, or z).
These transformations change both the position and orientation of objects.
Matrix representation is used to perform these transformations efficiently.

Advantages:

- Essential for 3D modeling and animation
- Allows precise control over objects in 3D space

Disadvantages:

- More complex than 2D transformations
- Requires careful handling of matrix calculations

C Code

```
 #include <stdio.h>
#include <math.h>

int main() {
   int choice, axis;
   float x, y, z;
   float tx, ty, tz;
   float sx, sy, sz;
   float angle;
   float x_new, y_new, z_new;

   printf("Enter original point (x y z): ");
   scanf("%f %f %f", &x, &y, &z);

   printf("\n3D Transformations Menu:");
   printf("\n1. Translation");
   printf("\n2. Scaling");
   printf("\n3. Rotation");
   printf("\nEnter your choice: ");
   scanf("%d", &choice);

   switch (choice) {

      case 1: // Translation
         printf("Enter translation factors (tx ty tz): ");
```

```
    scanf("%f %f %f", &tx, &ty, &tz);
    x_new = x + tx;
    y_new = y + ty;
    z_new = z + tz;
    break;

case 2: // Scaling
    printf("Enter scaling factors (sx sy sz): ");
    scanf("%f %f %f", &sx, &sy, &sz);
    x_new = x * sx;
    y_new = y * sy;
    z_new = z * sz;
    break;

case 3: // Rotation
    printf("\nRotate about:");
    printf("\n1. X-axis");
    printf("\n2. Y-axis");
    printf("\n3. Z-axis");
    printf("\nEnter axis choice: ");
    scanf("%d", &axis);

    printf("Enter angle (in degrees): ");
    scanf("%f", &angle);
    angle = angle * (3.14159 / 180); // degrees to radians
```

```
        if (axis == 1) { // X-axis
            x_new = x;
            y_new = y * cos(angle) - z * sin(angle);
            z_new = y * sin(angle) + z * cos(angle);
        } else if (axis == 2) { // Y-axis
            x_new = x * cos(angle) + z * sin(angle);
            y_new = y;
            z_new = -x * sin(angle) + z * cos(angle);
        } else if (axis == 3) { // Z-axis
            x_new = x * cos(angle) - y * sin(angle);
            y_new = x * sin(angle) + y * cos(angle);
            z_new = z;
        } else {
            printf("Invalid axis choice!");
            return 0;
        }
        break;

    default:
        printf("Invalid choice!");
        return 0;
}

printf("\nNew point after transformation:");
printf("\n(%.2f , %.2f , %.2f)", x_new, y_new, z_new);
```

```
    return 0;
}
```